Ejercicios de Evaluación

Consideraciónes Iniciales

- Ejercicios sobre
  - Refactoring
  - Patrones
- Preguntas:
  - Definiciones (memoria)
  - Definiciones aplicadas a casos
  - Solución de casos con refactorings o con patrones
    - Leer con detenimiento el ejercicio
    - Contestar de manera acorde
      - Listar
      - Justificar

1. Un mercado electrónico siempre se encuentra en uno de estos Modos durante su operación: Abierto, Cerrado, Suspendido. Se conocen las condiciones por las que un mercado cambia de un modo a otro.

   Opciones: Strategy, Decorator, State, Builder.

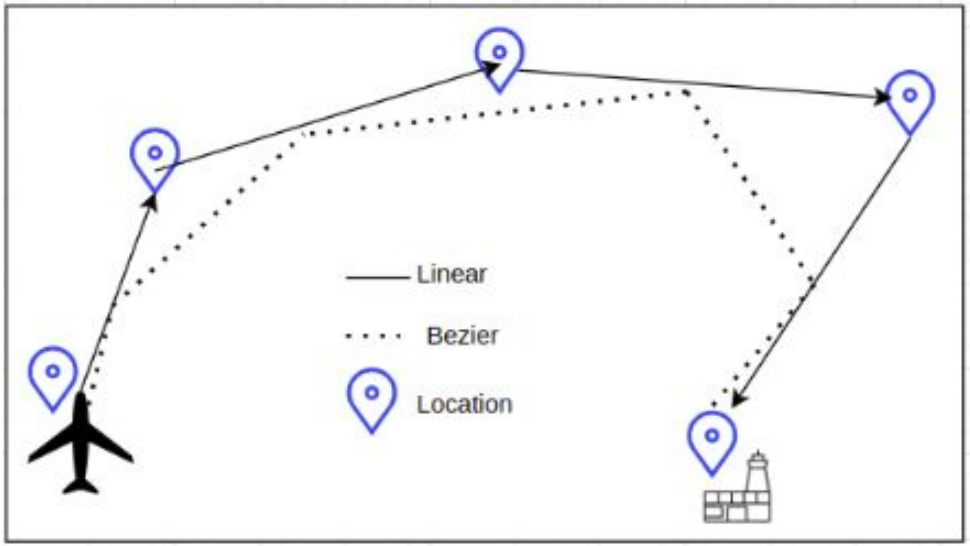
2. Un sistema de mensajería permite agregar varias funcionalidades adicionales a los mensajes en tiempo de ejecución, como cifrado, compresión o firma digital.

   Opciones: Strategy, Decorator, State, Builder.

3. Un sistema de menú de hamburguesas que brinda al usuario opciones para poder formar su hamburguesa personalizada. El sistema permite elegir tipo de paty (de carne o de lentejas), ingredientes que acompañan al paty (tomate, cebolla, panceta, pepino) y un acompañamiento (papas fritas, aros de cebolla)

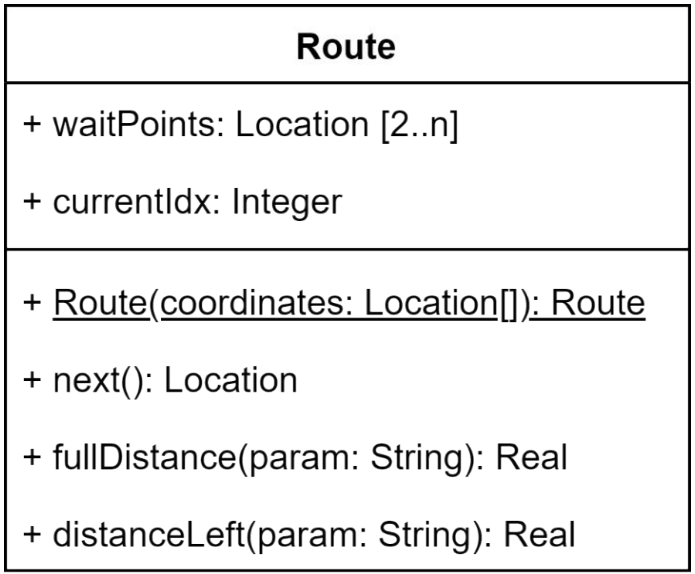
   Opciones: Adapter, Decorator, State, Builder.

Route



Route
+ waitPoints: Location [2..n] + currentIdx: Integer
+ Route(coordinates: Location[]): Route + next(): Location + fullDistance(param: String): Real + distanceLeft(param: String): Real

Route



`next`: retorna el siguiente punto del recorrido.
`fullDistance`: tiene como parámetro un String que denota un tipo de interpolación (Linear o Bezier). Esta operación retorna la distancia total del recorrido según el parámetro, que puede tener valores “Linear” o “Bezier”.
`distanceLeft`: tiene como parámetro un String que denota un tipo de interpolación (Linear o Bezier). Esta operación retorna la distancia que falta recorrer según el parámetro, que puede tener valores “Linear” o “Bezier”.

```
1  public double fullDistance(String param) {
2     double res = 0;
3     for (int i = 1; i < waitPoints.length; i++) {
4        if (param.equals("Linear"))
5           res += waitPoints[i].distanceTo(waitPoints[i-1]);
6        else
7           res += (waitPoints[i].middle(waitPoints[i-1])).distanceTo(waitPoints[i-1]);
8     }
9     return res; }
```

```
11 public double distanceLeft(String param) {
12    double res = 0;
13    for (int i = currentIdx + 1; i < waitPoints.length; i++) {
14       if (param.equals("Linear"))
15          res += waitPoints[i].distanceTo(waitPoints[i-1]);
16       else
17          res += (waitPoints[i].middle(waitPoints[i-1])).distanceTo(waitPoints[i-1]);
18    }
19    return res; }
```

```
21 public Location next() {
22    return currentIdx < waitPoints.length ? waitPoints[currentIdx++]
23       : waitPoints[waitPoints.length - 1];  }
```

1. Liste los métodos de instancia que implementaría la clase Location.
   //Simple y ayuda a pensar en la Location

2. Liste los refactorings que aplicaría para eliminar el **código repetido**.
   //No pide el máximo número de refactorings solo los necesarios
   //La clave es qué cuál es el **código repetido**
   //No pide los refactorings pide una lista (no vacía) de nombres de refactorings aplicados al código del ejercicio.

3. Transcriba el código resultante después de aplicar todos los refactorings.
   //No pide los pasos intermedios…solo el código resultante

Código Repetido

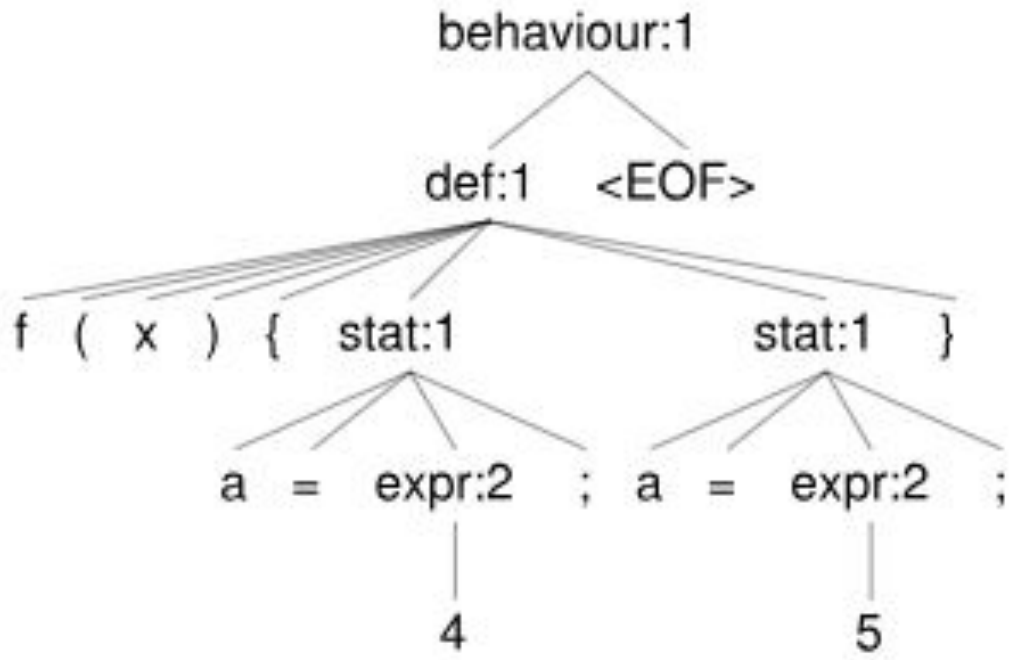
```
1  public double fullDistance(String param) {
2     double res = 0;
3     for (int i = 1; i < waitPoints.length; i++) {
4        if (param.equals("Linear"))
5           res += waitPoints[i].distanceTo(waitPoints[i-1]);
6        else
7           res += (waitPoints[i].middle(waitPoints[i-1])).distanceTo(waitPoints[i-1]);
8     }
9     return res; }
```

```
11 public double distanceLeft(String param) {
12    double res = 0;
13    for (int i = currentIdx + 1; i < waitPoints.length; i++) {
14       if (param.equals("Linear"))
15          res += waitPoints[i].distanceTo(waitPoints[i-1]);
16       else
17          res += (waitPoints[i].middle(waitPoints[i-1])).distanceTo(waitPoints[i-1]);
18    }
19    return res; }
```

```
21 public Location next() {
22    return currentIdx < waitPoints.length ? waitPoints[currentIdx++]
23       : waitPoints[waitPoints.length - 1];  }
```

Árboles de Parseo & Pseudocódigo

```
f(x) {
        a = 4;
        a = 5;
}
```



> Smell: La primer asignación no tiene efecto

Idea(v1.0):

Recorrer el abol.

Guardar en un ***set*** la vars asignadas

Eliminar del ***set*** vars que son leídas

Si al final el set es !vacio ⇒ smell

```
f(x) {
        a = 4;
        a = 5;
}
```

Idea*(v2.0):

Recorrer el abol.

Eliminar del ***set*** vars que son leídas

Guardar en un ***set*** la vars asignadas

Si al final el set es !vacio ⇒ smell

```
f(x) {
        a = 4;
        a = a + 5;
}
```

** Considerando que una asignación (=) tiene una var a la izquierda y una expresión a la derecha*

Idea(v2.0):

Recorrer el abol.

Eliminar del ***set*** vars que son leídas

Guardar en un ***set*** la vars asignadas

Si al final el set es !vacio ⇒ smell

PseudoCodigo(idea(v2.0))

//Considerando los árboles generados por la gramática de “pool”

// las vars son “expressions”

1. Recorrer el árbol
    a. Por cada referencia a variable
        i. Si no está a la izquierda de una asignación eliminar de S:Set
        ii. Si está a la izquierda de una asignación agregar a S:Set
2. Si S:set no es vacío existen asignacion a variables sin efecto.

Consideraciónes finales

- Ejercicios sobre
  - Refactoring
  - Patrones
- Preguntas:
  - Definiciones (memoria)
  - Definiciones aplicadas a casos
  - Solución de casos con refactorings o con patrones
    - Leer con detenimiento el ejercicio
    - Contestar de manera acorde
      - Listar
      - Justificar